

Partner API — Manual de integración

Versión 1.15

API para integrar tu sitio (landing) con la plataforma de casino: alta de jugadores, login único, gestión de contraseñas, consulta de saldo y carga/retiro de fichas.

1. Conceptos básicos

- **Base URL:** te la entregamos junto con tus credenciales. Ejemplo: `https://api-lbvip.com/api/v1`
- **Autenticación:** todas las requests llevan el header `X-API-Key`.
- **Formato:** JSON en el body (`Content-Type: application/json`) y JSON en las respuestas.
- **Ámbito:** tu key opera sobre **tus** jugadores (los creados con ella). No podés ver ni operar jugadores de otros.

Cómo obtener la key (autoservicio)

La key la genera **tu agente** desde su panel: menú **Clientes de API** → **Nuevo cliente**. Al crearla:

- Los jugadores creados con esa key quedan **bajo la cuenta del agente que la creó**.
- Los depósitos salen del **saldo de ese agente** (y los retiros vuelven a él), igual que una carga manual.
- La key se muestra **una sola vez** al crearla — guardala en el momento.
- Desde el mismo panel el agente puede **desactivarla, regenerarla o eliminarla** cuando quiera, sin pedirle nada a nadie.

```
X-API-Key: pk_a1b2c3d4_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json
Accept: application/json
```

Whitelist de IPs (opcional)

Tu key puede restringirse a **IPs fijas**: si tu sistema llama siempre desde las mismas IPs, pedile al operador que las cargue (o cargalas desde el panel, botón de IP en Clientes de API). Con la whitelist activa, una request desde otra IP recibe `401` con `code: "ip_not_allowed"` (el mensaje incluye la IP detectada, útil para registrar la correcta). Lista vacía = sin restricción. La key sigue siendo obligatoria siempre — esto es una defensa extra.

Seguridad de la key

- La key es **secreta**: usala **solo desde tu servidor** (server-to-server). Nunca en el navegador, ni en una app, ni en el código fuente del frontend.
- Si sospechás que se filtró, avisanos y la regeneramos en el momento (la vieja deja de funcionar al instante).

Límite de uso

- **60 requests por minuto.** Si te pasás, recibís `429 Too Many Requests` — esperá y reintentá.

2. Endpoints

2.1 Crear jugador

Crea un jugador en la plataforma, vinculado a tu cuenta.

```
POST /players
```

Campo	Tipo	Requerido	Notas
username	string	✓	3-18 caracteres, solo letras/números/guion bajo. Único.
password	string	✓	mínimo 6 caracteres
email	string	—	
phone	string	—	
agent_id	int	—	Crea el jugador bajo un agente de tu red (ver 2.14 Listar agentes). Omitido = bajo el agente dueño de tu key (comportamiento por defecto).

agent_id (opcional). Si manejas varios agentes con una sola key, podés colgar cada jugador del agente que corresponda. El id tiene que ser un agente **de tu red** (el dueño de la key o alguien debajo suyo); cualquier otro valor devuelve `422 agent_not_allowed`. Obtené los ids válidos con `GET /agents` (2.14).

```
# Bajo el agente dueño de la key (por defecto)
curl -X POST https://api-lbvip.com/api/v1/players \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"username": "juan123", "password": "secreta1", "phone": "099111222"}'

# Bajo un agente específico de tu red
curl -X POST https://api-lbvip.com/api/v1/players \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"username": "juan123", "password": "secreta1", "agent_id": 42}'
```

Respuesta `201`:

```
{
  "player": {
    "username": "juan123",
    "email": null,
    "active": true,
    "balance": 0,
    "created_at": "2026-07-11T15:30:00-03:00"
  }
}
```

2.2 Validar credenciales

Verifica usuario y contraseña (para tu flujo de login).

```
POST /players/validate
```

```
curl -X POST https://api-lbvip.com/api/v1/players/validate \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"username": "juan123", "password": "secret1"}'
```

Respuesta **200** :

```
{ "valid": true, "player": { "username": "juan123", "balance": 1500, "...": "..." } }
```

Con contraseña incorrecta: { "valid": false, "player": null } (también **200**).

2.3 Consultar jugador (datos + saldo)

```
GET /players/{username}
```

Respuesta **200** :

```
{ "player": { "id": 5, "username": "juan123", "email": null, "active": true, "balance": 1500, "crea
```

Si la plataforma tiene **Rollover y Bonos** habilitado (ver 2.8), el `player` incluye además el desglose `wagering` :

```
{
  "player": {
    "username": "juan123",
    "balance": 11000,
    "wagering": {
      "balance": 11000,
      "locked": 11000,
      "bonus_locked": 1000,
      "available": 0,
      "claimable": [ { "id": 42, "amount": 1000, "wagered": true } ],
      "claimable_total": 1000,
      "requirements": [
        { "id": 41, "kind": "deposit", "locked": 10000, "multiplier": 1, "target": 10000, "progress": 0 },
        { "id": 42, "kind": "bonus", "locked": 1000, "multiplier": 10, "target": 10000, "progress": 0 }
      ]
    }
  }
}
```

- `balance` = saldo **jugable** total (el de siempre; incluye el bono).
- `available` = lo único **retirable** hoy (`balance` – bloqueado por requisitos).
- `bonus_locked` = cuánto de lo bloqueado es bono (útil si mostrás "Bono" aparte, o para lógica tipo "los bonos no suman VIP").
- `requirements` = objetivos de apuesta pendientes, con progreso (`pct` para tu barra de progreso).
- `claimable` = **bonos ganados que el jugador todavía no reclamó**. Un bono que cumple su objetivo (o que se otorgó con `0`, sin objetivo) **no se libera solo**: queda bloqueado hasta que el jugador lo reclama, para que se entere de que lo ganó. En el casino eso es el regalito del header; si lo mostrás en tu sitio, avísale que lo tiene pendiente. `wagered` distingue el que ganó jugando (`true`) del que se le regaló sin rollover (`false`).

2.4 Cambiar contraseña

Cambia la contraseña del jugador **sin necesitar su sesión** (vos ya lo autenticaste en tu sitio). Cierra todas sus sesiones abiertas en la plataforma.

```
PUT /players/{username}/password
```

```
curl -X PUT https://api-lbvip.com/api/v1/players/juan123/password \
-H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
-d '{"password": "nueva123"}'
```

Respuesta 200: { "message": "Contraseña actualizada." }

Usalo para mantener la contraseña **sincronizada**: cuando el usuario la cambia en tu sitio, llama este endpoint y queda igual en la plataforma.

2.5 Depositar fichas

Acredita saldo al jugador.

```
POST /players/{username}/deposit
```

Campo	Tipo	Requerido	Notas
amount	number	✓	mayor a 0. Unidad = pesos (unidad mayor, NO centavos) ; admite decimales. Ej: <code>100.50</code> = 100,50. Es la misma unidad del <code>balance</code> que devolvemos.
reference	string	✓	tu ID único de la operación (ver idempotencia abajo)
description	string	—	texto libre para el historial
promo_code	string	—	código promocional del jugador (ver reglas automáticas en 2.8)
no_bonus	boolean	—	<code>true</code> = este depósito no hereda ninguna regla automática de bono (ver 2.8)

```
curl -X POST https://api-lbvip.com/api/v1/players/juan123/deposit \
-H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
-d '{"amount": 500, "reference": "dep-000123"}'
```

Respuesta 201 :

```
{
  "duplicate": false,
  "operation": { "reference": "dep-000123", "type": "deposit", "ledger_id": 98765, "created_at": "...",
  "balance": 2000
}
```

Idempotencia (importante)

La `reference` es **tu número de operación** y tiene que ser **único por operación**. Si repetís una request con la misma `reference` (por ejemplo, por un timeout y retry), **no se duplica la carga**: recibís `200` con `"duplicate": true` y los datos de la operación original.

Regla de oro: ante un error de red o timeout, **reintentá con la misma `reference`**. Nunca generes una nueva para el mismo depósito.

2.6 Retirar fichas

Debita saldo del jugador. Mismos campos que el depósito.

```
POST /players/{username}/withdraw
```

Respuesta 201: igual formato que depósito, con el saldo resultante.

Si el jugador no tiene saldo suficiente: 422 con código `insufficient_funds`.

Si la plataforma tiene **Rollover y Bonos** habilitado y el jugador tiene requisitos pendientes, solo puede retirar su **disponible** (`wagering.available`): un retiro mayor devuelve 422 con código `rollover_locked` (ver 2.8).

2.7 Login único (entrar a la plataforma sin re-login)

Devuelve un **link de acceso directo**: redirigí al usuario a esa URL y entra a la plataforma ya logueado.

```
POST /players/{username}/session
```

Campo	Tipo	Requerido	Notas
<code>password</code>	string	—	Opcional. Como vos ya autenticaste al jugador en tu sitio, con la API key + username alcanza. Si la enviás, se valida (útil si querés reusar este endpoint en el primer login del jugador).
<code>embed</code>	boolean	—	Opcional (modo embebido). Con <code>true</code> , el <code>redirect_url</code> sale con <code>&embed=1</code> y el casino oculta sus controles de sesión (Cerrar sesión / Iniciar sesión / Registrarse) en esa pestaña/iframe: el cambio de usuario pasa SOLO por tu app y nunca quedan cruzados el usuario de tu app y el del juego.

```
# Sin contraseña (recomendado para login único: ya lo autenticaste vos)
curl -X POST https://api-lbvip.com/api/v1/players/juan123/session \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" -d '{}'
```

Respuesta 200:

```
{
  "token": "1234|abcdef...",
  "redirect_url": "https://looneybet.vip/auth/sso?code=Xy7...un-solo-uso...",
  "logout_url": "https://looneybet.vip/logout"
}
```

El `redirect_url` lleva un **código de un solo uso** (vence en 60 segundos) — no el token. Usalo apenas lo recibís. El `token` del campo aparte sigue disponible si tu integración lo usa directo.

Flujo recomendado: 1. El usuario se loguea en tu sitio. 2. Cuando toca "Jugar", tu servidor llama a `/session`. 3. Redirigís el navegador del usuario a `redirect_url` → entra directo, sin segundo login.

El token es de un solo contexto de sesión: pedí uno nuevo cada vez que el usuario vaya a entrar. No lo guardes.

Semántica de sesión (multi-usuario en el mismo navegador/iframe): - El `redirect_url` (`/auth/sso`) **SIEMPRE reemplaza** la sesión que hubiera en ese navegador: primero la cierra y recién después canjea el código. Si el canje falla (código vencido o **ya usado** — es de un solo uso: ojo con prefetch/doble carga del iframe), el navegador queda **deslogueado** — nunca con el usuario anterior. - `logout_url` (GET `/logout`): cargala en el iframe para cerrar la sesión activa (revoca el token server-side). Útil como cinturón antes de inyectar el SSO de otro usuario, aunque con el reemplazo automático de arriba ya no es imprescindible. - Para el caso "un celular, varios usuarios": pedí `/session` con `"embed": true` y usá siempre ese `redirect_url` — el jugador no puede cerrar sesión ni loguearse desde adentro del casino, así el usuario de tu app y el del juego jamás se cruzan.

2.8 Depósitos con rollover y bono

Disponible solo si la plataforma tiene el feat **Rollover y Bonos** habilitado (lo activa el operador del casino). Si mandás estos campos con el feat apagado, recibís `422 feature_disabled` y **no se mueve plata**.

El depósito de 2.5 acepta tres campos opcionales que le agregan **condiciones de retiro** a la carga. El jugador **puede jugar todo su saldo normalmente** — lo que queda bloqueado hasta cumplir el objetivo de apuestas es únicamente **el retiro**.

Campo	Tipo	Requerido	Notas
multiplier	integer	—	Rollover de la carga: 0 , 1 , 2 , 5 o 10 . El jugador debe apostar $\text{amount} \times \text{multiplier}$ para poder retirar. 0 (o ausente) = carga libre, sin requisito.
bonus_percent	integer	—	Bono de regalo: 1–100 (% del amount). Se acredita como saldo jugable ya mismo , en una operación contable separada. Sale del saldo del agente , igual que la carga. Excluyente con bonus_amount .
bonus_amount	number	—	Bono de monto fijo (en pesos, en lugar del %). Mismo comportamiento que bonus_percent ; sujeto a los límites min/max que configure el operador (422 bonus_out_of_range si se sale). Excluyente con bonus_percent .
bonus_multiplier	integer	Con el bono	Objetivo propio del bono: 0 , 2 , 5 , 10 , 20 o 40 . 0 = sin rollover : regalo directo — queda disponible/retirable al instante, sin reclamo, y no pisa el bono en curso. El bono se vuelve retirable al apostar $\text{bono} \times \text{bonus_multiplier}$. Sugerencia estándar: $100 \div \text{bonus_percent}$ (ej.: bono 10% → 10x → el objetivo del bono queda igual al monto de la carga).

Reglas importantes:

- **Orden de consumo del saldo:** jugando se gasta primero el **real bloqueado**, después el **bono** y por último el **disponible** (lo retirable queda protegido hasta el final). Las ganancias vuelven al saldo del que se venía jugando.
- Las **mismas apuestas** progresan **todos** los objetivos activos a la vez (carga y bono avanzan juntos). *Excepción configurable por el operador del casino:* en modo "**cada saldo con su propia plata**", cada apuesta progresa solo el objetivo del saldo del que salió — lo jugado con saldo bloqueado avanza el de la carga, lo jugado con bono avanza el del bono, y **lo jugado con el disponible no progresa nada**.
- **Bono sobre bono:** otorgar un bono a un jugador que ya tiene un bono activo **pisa el anterior** — se debita lo que le quede del saldo del bono viejo ("Cancelación de bono") y rige la configuración del nuevo.
- **Carga con rollover sobre rollover pendiente:** el objetivo anterior se **reemplaza** por uno nuevo cuya base es *lo que quedaba bloqueado + la carga nueva*, con el multiplicador de la última carga (acá no se debita nada: la plata queda, solo se re-consolida el objetivo y la barra arranca de 0).
- Si el jugador **pierde todo el saldo** jugando, los requisitos pendientes se cancelan solos (y una carga nueva sobre saldo en cero arranca limpia).
- El progreso se mide sobre lo **apostado bruto** en los juegos, desde el momento del depósito (las apuestas anteriores no cuentan).
- `reference` sigue siendo tu llave de idempotencia: el retry de un depósito con bono **no** duplica ni la carga ni el bono.


```
# Carga de $10.000 con rollover 1x + bono 10% con objetivo 10x
curl -X POST https://api-lbvip.com/api/v1/players/juan123/deposit \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"amount": 10000, "reference": "dep-000124", "multiplier": 1, "bonus_percent": 10, "bonus_mult": 10}'
```

Respuesta 201 :

```
{
  "duplicate": false,
  "operation": { "reference": "dep-000124", "type": "deposit", "ledger_id": 98801, "created_at": "2023-01-01T12:00:00Z" },
  "balance": 11000,
  "wagering": {
    "deposit_requirement_id": 41,
    "bonus": { "status": "ok", "amount": 1000, "multiplier": 10, "ledger_id": 98802, "requirement_id": 42 },
    "breakdown": {
      "balance": 11000, "locked": 11000, "bonus_locked": 1000, "available": 0,
      "requirements": [
        { "id": 41, "kind": "deposit", "target": 10000, "progress": 0, "remaining": 10000, "pct": 0 },
        { "id": 42, "kind": "bonus", "target": 10000, "progress": 0, "remaining": 10000, "pct": 0 }
      ]
    }
  }
}
```

- `bonus.status` puede ser `"failed"` en un caso excepcional: la **carga quedó acreditada** pero el bono no pudo otorgarse. No reintentes el depósito completo (la `reference` te va a devolver `duplicate`) — avisanos y lo resolvemos.
- En un replay idempotente (`"duplicate": true`) el campo `wagering` viene reconstruido con los mismos `requirement_id` del primer intento.

Retiro bloqueado por rollover (POST `/withdraw` con requisitos pendientes):

```
{
  "error": { "code": "rollover_locked", "message": "Retiro bloqueado por rollover: tenés $0,00 disponibles" },
  "wagering": { "balance": 11000, "locked": 11000, "available": 0, "requirements": [ "... " ] }
}
```

Buenas prácticas del feat:

- Antes de ofrecer "Retirar" en tu sitio, consultá `GET /players/{username}` y mostrá `wagering.available` (no `balance`).
- Para regalos recurrentes (reintegros, rachas, promos), usá `bonus_percent` o un depósito con `multiplier` bajo (1) — así el regalo pide al menos una vuelta de juego antes de poder retirarse.

- Si tu lógica distingue "plata real" de "bonos" (ej. niveles VIP), usá las operaciones: la carga y el bono llegan como **dos** operaciones contables separadas (el ledger del bono es el `wagering.bonus.ledger_id`).

Reglas automáticas de bono (campañas del agente)

El agente puede configurar **campañas** desde su panel (Solicitudes → Cargas): *bono de bienvenida* (al registrarse), *1er depósito* y *códigos promocionales*. Afectan a la API así:

- **Herencia en el depósito:** un `POST /deposit` sin params de bono (`bonus_percent` / `bonus_amount`) hereda automáticamente la regla que corresponda al jugador — un `promo_code` válido primero, si no la de *1er depósito* (primera carga real del jugador desde que la regla está vigente, una vez por jugador). El bono aparece en la respuesta (`wagering.bonus`) igual que si lo hubieras mandado vos.
- **Tus params siempre ganan:** si mandás `bonus_percent` / `bonus_amount` explícitos, la regla no se aplica.
- **Opt-out:** mandá `"no_bonus": true` para que ese depósito no herede nada.
- **promo_code** : pasá el código que el jugador ingresó en tu sitio. Si es inválido, vencido o ya usado, **no es error**: el depósito se acredita normal, sin bono de código (puede caer en la regla de 1er depósito si aplica).
- **Bienvenida:** se otorga solo, al crear el jugador (`POST /players` incluido), como bono suelto de monto fijo. Sale del saldo del agente; si no le alcanza, el alta sale igual y el bono simplemente no se otorga.
- Los montos definidos por regla **no** están sujetos a los límites min/max del bono fijo (esos rigen solo para tus params explícitos).
- Consultá las reglas vivas de tu red en `GET /config` → `bonus.auto_rules` (2.13).

2.9 Bono suelto (sin depósito)

Disponibile solo si el operador del casino habilitó además el **bono suelto**. Con el feat apagado: `422 feature_disabled`.

Acredita un bono de **monto fijo** con su rollover, sin depósito asociado — ideal para reintegros, rachas y promos recurrentes. La plata sale del saldo del agente y el bono queda bloqueado hasta cumplir `amount × multiplier` (con las mismas reglas de 2.8: pisa al bono anterior, se cancela al quedar en cero, etc.).

```
POST /players/{username}/bonus
```

Campo	Tipo	Requerido	Notas
amount	number	Sí	monto del bono, en pesos. Sujeto a los límites min/max del operador (422 bonus_out_of_range).
multiplier	integer	Sí	objetivo del bono: 0 , 2 , 5 , 10 , 20 o 40 (apostar amount × multiplier para liberarlo). 0 = sin rollover: regalo directo — disponible/retirable al instante, sin reclamo, y no pisa el bono en curso.
reference	string	Sí	tu ID único de operación — misma idempotencia que el depósito (retry con la misma reference no duplica).

```
# Reintegro diario: bono fijo de $200 con objetivo 10x
curl -X POST https://api-lbvip.com/api/v1/players/juan123/bonus \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"amount": 200, "multiplier": 10, "reference": "cashback-20260729-juan123"}'
```

Respuesta 201 :

```
{
  "duplicate": false,
  "operation": { "reference": "cashback-20260729-juan123", "type": "bonus", "ledger_id": 98910, "cr
  "balance": 1700,
  "wagering": {
    "bonus": { "status": "ok", "amount": 200, "multiplier": 10, "ledger_id": 98910, "requirement_id
    "breakdown": { "balance": 1700, "locked": 200, "bonus_locked": 200, "available": 1500, "require
  }
}
```

2.10 Netwin (GGR) del jugador

Cuánto apostó, cuánto se le pagó y cuánto quedó para la casa en un período. Es el dato para liquidar **reembolsos por pérdida y comisiones de referidos**.

```
GET /players/{username}/stats?from=2026-07-01 00:00:00&to=2026-07-31 23:59:59
```

Campo	Tipo	Requerido	Notas
from	string	—	Y-m-d (arranca 00:00:00) o Y-m-d H:i:s . Default: hace 30 días.
to	string	—	Y-m-d (termina 23:59:59) o Y-m-d H:i:s . Default: ahora.

Respuesta 200 :

```
{
  "player": { "username": "juan123", "id": 5 },
  "from": "2026-07-01 00:00:00",
  "to": "2026-07-31 23:59:59",
  "totals": { "bets_count": 89, "wagered": 248830, "payout": 59078, "netwin": 189752 },
  "categories": {
    "casino": { "bets_count": 89, "wagered": 248830, "payout": 59078, "netwin": 189752 },
    "sports": { "bets_count": 0, "wagered": 0, "payout": 0, "netwin": 0 }
  }
}
```

- `wagered` = total apostado · `payout` = total pagado en premios · `netwin` = `wagered` - `payout` .
- **Ojo con el signo:** `netwin` POSITIVO significa que ganó la casa, o sea que el jugador **perdió** — que es justo la base del reembolso. Negativo = el jugador ganó en el período.
- `bets_count` cuenta jugadas, no transacciones (un giro con premio es una).
- El rango **respetar la hora** y se evalúa en horario de Argentina, así que cortar a la medianoche argentina sale natural. Máximo **92 días** por consulta (`invalid_range` si te pasás).
- Los números son **en vivo**: incluyen lo jugado hace un minuto. Pueden diferir por unos minutos de los reportes del panel, que leen un rollup que se recalcula cada ~10 min.
- Un jugador sin actividad devuelve `200` con todo en cero (no `404`).

2.11 Netwin de varios jugadores (batch)

Mismo dato que 2.10 para hasta **100 jugadores** en un request — pensado para liquidar todos los referidos de una sin comerse el límite de 60 requests/minuto.

```
POST /players/stats/batch
```

Campo	Tipo	Requerido	Notas
<code>usernames</code>	array	Sí	1 a 100 usernames.
<code>from</code>	string	—	Igual que 2.10.
<code>to</code>	string	—	Igual que 2.10.

```
curl -X POST https://api-lbvip.com/api/v1/players/stats/batch \
-H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
-d '{"usernames": ["juan123", "ana456"], "from": "2026-07-01", "to": "2026-07-31"}'
```

Respuesta 200 :

```
{
  "from": "2026-07-01 00:00:00",
  "to": "2026-07-31 23:59:59",
  "players": [
    { "username": "juan123", "id": 5, "totals": { "bets_count": 89, "wagered": 248830, "payout": 59
      "categories": { "casino": { "...": "..." }, "sports": { "...": "..." } } }
  ],
  "not_found": ["ana456"]
}
```

- `not_found` lista los usernames que no existen **o que no son tuyos**: no distinguimos entre los dos casos para no filtrar quién juega en otra cuenta.

2.12 Reclamar el bono cumplido

Desde la 1.7 un bono que cumple su objetivo **no se libera solo**: queda *a reclamar*. (Salvo que el operador haya configurado la acreditación directa — mirá `bonus.claim_required` en 2.13. El bono `0`, sin rollover, nunca pasa por acá: se acredita directo.) En el casino lo reclama el jugador tocando el regalito del header, pero si tus jugadores operan desde tu sitio y no entran a la plataforma, reclamás vos por acá.

```
POST /players/{username}/bonus/claim
```

Campo	Tipo	Requerido	Notas
<code>requirement_id</code>	integer	—	Reclamar UNO puntual. Sin esto se reclaman todos los pendientes.

Respuesta 200 :

```
{
  "claimed": [ { "id": 42, "amount": 1000 } ],
  "amount": 1000,
  "wagering": { "balance": 1500, "available": 1500, "bonus_locked": 0, "claimable": [], "...": "..."
}
```

- Los pendientes salen de `wagering.claimable` en `GET /players/{username}` (2.3): ahí ves cuánto tiene sin reclamar y desde cuándo.
- **Es idempotente**: si no quedaba ninguno devuelve `200` con `amount: 0` y `claimed: []`. Reintentar por timeout no acredita dos veces.
- Reclamar **no mueve plata**: el bono ya estaba en el saldo del jugador, lo que hace es destrabarlo (pasa a `available`, o sea retirable).
- Si el jugador lo reclamó desde el casino un segundo antes, tu llamada devuelve `amount: 0` — no es un error.

2.13 Configuración del sitio

Lo que necesitás saber ANTES de mandar una operación: qué está habilitado, qué multiplicadores son válidos y los límites del bono de monto fijo. Consultalo al iniciar y cacheálo; cambia solo cuando el operador toca la configuración.

```
GET /config
```

Respuesta 200 :

```
{
  "rollover": { "enabled": true, "multipliers": [0, 1, 2, 5, 10] },
  "bonus": {
    "enabled": true,
    "standalone_enabled": true,
    "multipliers": [0, 2, 5, 10, 20, 40],
    "percents": [10, 20, 50],
    "fixed_min": 1,
    "fixed_max": 100000,
    "claim_required": true,
    "auto_rules": [
      { "type": "first_deposit", "name": "Promo 50", "code": null, "mode": "percent", "percent": 50 },
      { "type": "promo_code", "name": "Verano", "code": "VERAN02026", "mode": "fixed", "percent": n
    ]
  }
}
```

- `claim_required` en `false` significa que **este sitio acredita los bonos solos**: al cumplir el objetivo (o al otorgar un bono `0`) la plata queda disponible sin pasar por el reclamo de 2.12. En ese modo `wagering.claimable` siempre viene vacío y llamar al reclamo devuelve `amount: 0`.
- `fixed_min` / `fixed_max` son los límites de `bonus_amount` (2.8 y 2.9). `0` = **sin límite**. Validá contra esto y le avisás al cajero antes de comerte un `bonus_out_of_range`.
- `percents` son los porcentajes que cargó el operador para `bonus_percent`. Vacío = no definió ninguno.
- `standalone_enabled` en `false` significa que el bono suelto (2.9) devuelve `feature_disabled`.
- `auto_rules` son las **reglas automáticas vivas** que rigen para tu red (ver 2.8): las del agente dueño de tu key **más las globales del sitio** (`scope: "global"` — aplican como fallback cuando el agente no tiene regla propia de esa categoría). Un depósito sin params de bono hereda la que corresponda. `type` es `welcome` (al registrarse), `first_deposit` o `promo_code`. Útil para mostrar las promos vigentes en tu sitio.

2.14 Listar agentes de tu red

Devuelve los agentes que tenés **debajo** de tu cuenta, para elegir bajo cuál crear un jugador (`agent_id` en 2.1). Solo ves **tu** red: nunca los agentes de otro.

```
GET /agents
```

```
curl https://api-lbvip.com/api/v1/agents \
-H "X-API-Key: $API_KEY"
```

Respuesta 200 :

```
{
  "agents": [
    { "id": 42, "name": "agente_norte" },
    { "id": 57, "name": "agente_sur" }
  ]
}
```

- La lista incluye a **todos** los agentes de tu subárbol (hijos, nietos, etc.), ordenados por nombre.
- Si tu key no tiene agentes por debajo, la lista viene **vacía** — creá los jugadores sin `agent_id` (van bajo tu cuenta).
- El `id` es el que se pasa como `agent_id` al crear un jugador (2.1).

2.15 Listar cargas y retiros

Listado de las **solicitudes de carga y retiro** (las que los jugadores crean desde el sitio y los cajeros aprueban/rechazan), para conciliar desde tu sistema. Requiere que el operador habilite la función para tu sitio — si no está habilitada, responde `403` aunque la key sea válida.

```
GET /chip-requests
```

Parámetros (query, todos opcionales):

Parámetro	Valores	Descripción
type	deposit withdrawal	solo cargas o solo retiros
status	pending accepted rejected	filtrar por estado
user_id	entero	solicitudes de UN jugador
from / to	fecha (YYYY-MM-DD)	rango por fecha de creación (to incluye el día completo)
page / per_page	enteros	paginado; per_page máximo 100 (default 50)

```
# Todas las cargas pendientes de hoy
curl "https://api-lbvip.com/api/v1/chip-requests?type=deposit&status=pending&from=2026-08-29" \
-H "X-API-Key: $API_KEY"
```

Respuesta 200 :

```
{
  "data": [
    {
      "id": 1382,
      "type": "deposit",
      "status": "accepted",
      "amount": 10000.0,
      "credited_amount": 11000.0,
      "currency": "ARS",
      "promo_code": "BIENVENIDA10",
      "created_at": "2026-08-29T14:03:21-03:00",
      "processed_at": "2026-08-29T14:05:40-03:00",
      "user": { "id": 9000123, "name": "jugador55" },
      "agent": { "id": 42, "name": "agente_norte" }
    }
  ],
  "meta": { "page": 1, "per_page": 50, "total": 1, "last_page": 1 }
}
```

- agent es el **padre directo** del jugador (a qué agente/cajero pertenece).
- credited_amount puede diferir de amount en cargas con bono/promo; viene null si no aplica.
- processed_at es cuándo se aprobó/rechazó; null mientras está pending .
- Orden: más recientes primero (id descendente).
- **Ámbito:** igual que el resto de la API — tu key ve las solicitudes de **tu** red (una key de operador ve todo el sitio).

- Para conciliación por polling: guardá el último `id` visto y consultá periódicamente; los `id` son crecientes.

3. Errores

Formato estándar:

```
{ "error": { "code": "insufficient_funds", "message": "Saldo insuficiente." } }
```

HTTP	code	Cuándo
401	unauthorized	key inválida, inactiva o ausente
401	invalid_credentials	contraseña incorrecta (en <code>/session</code>)
404	player_not_found	el jugador no existe o no es tuyo
422	agent_not_allowed	el <code>agent_id</code> al crear un jugador no existe o no pertenece a tu red
422	insufficient_funds	retiro mayor al saldo
422	rollover_locked	retiro mayor al disponible por rollover pendiente (el body incluye <code>wagering</code> con el detalle)
422	feature_disabled	mandaste <code>multiplier</code> /bono y el feat correspondiente no está habilitado en la plataforma
422	bonus_out_of_range	el bono de monto fijo está fuera de los límites min/max configurados por el operador
422	invalid_range	el rango de fechas de las stats es inválido o supera los 92 días
422	—	error de validación (formato Laravel: <code>{ "message": "...", "errors": {...} }</code>)
429	—	superaste las 60 req/min
503	wallet_not_configured	mantenimiento del servicio de saldo — reintentá más tarde con la misma <code>reference</code>

4. Buenas prácticas

- **Todo server-to-server:** tu backend llama a la API; el navegador del usuario solo ve el `redirect_url` del login único.
- **Guardá la `reference`** de cada depósito/retiro en tu base — es tu comprobante y tu llave de reintento.
- **Reintentos:** ante `5xx` o timeout, reintentá con backoff (ej. 2s, 5s, 15s) y la **misma `reference`**.
- **No asumas el saldo:** usá el `balance` que devuelve cada operación o consultá `GET /players/{username}`.

- El header `Accept: application/json` evita respuestas HTML en errores.
-

5. Soporte

- Recibís del agente: **Base URL + API key** (una sola vez — guardala segura).
 - **Regenerar/desactivar la key:** lo hace tu agente al instante desde su panel (**Cientes de API**), sin ticket.
 - Para reportar un problema o pedir un endpoint nuevo, contactanos por el canal de soporte habitual.
-

Historial de cambios

Versión	Fecha	Cambios
1.13	2026-08-23	Reglas globales del sitio: el operador puede definir reglas para TODOS los agentes (fallback cuando el agente no tiene propia; la del agente siempre gana). <code>GET /config</code> → <code>bonus.auto_rules</code> ahora incluye las globales con <code>scope: "agent" "global"</code> . Sin cambios de contrato en los endpoints de operaciones.
1.12	2026-08-21	Reglas automáticas de bono (2.8): el depósito sin params de bono hereda la campaña del agente (código promocional > 1er depósito); nuevos campos <code>promo_code</code> y <code>no_bonus</code> en el depósito; bono de bienvenida al crear jugador (monto fijo, del saldo del agente); <code>GET /config</code> expone <code>bonus.auto_rules</code> . Los montos por regla no están sujetos a <code>fixed_min / fixed_max</code> .
1.11	2026-08-13	Jugadores por agente: <code>POST /players</code> acepta <code>agent_id</code> (opcional) para crear el jugador bajo un agente de tu red; nuevo <code>GET /agents (2.14)</code> lista tus agentes (id + nombre), scopeado a tu subárbol. Error nuevo <code>agent_not_allowed</code> .
1.10	2026-08-03	Bono 0 = regalo directo: vuelve a acreditarse disponible al instante (sin pasar por el reclamo) y ahora no pisa el bono en curso del jugador.
1.9	2026-07-31	Reclamo del bono por API (2.12): <code>POST /players/{username}/bonus/claim</code> , idempotente, para jugadores que no entran al casino. Configuración del sitio (2.13): <code>GET /config</code> expone <code>feats</code> , multiplicadores válidos y los límites min/max del bono fijo.
1.8	2026-07-31	Netwin (GGR) por jugador: nuevos <code>GET /players/{username}/stats (2.10)</code> y <code>POST /players/stats/batch (2.11)</code> , con rango por hora y separación casino/sports. <code>GET /players/{username}</code> pasa a incluir el <code>id</code> numérico del jugador. Error nuevo <code>invalid_range</code> .
1.7	2026-07-31	Bono a reclamar: los bonos cumplidos (y los <code>0</code> , sin rollover) ya no se liberan solos — quedan bloqueados hasta que el jugador los reclama. Nuevos campos <code>claimable / claimable_total</code> en el desglose <code>wagering</code> .
1.6	2026-07-29	Bono sin rollover: <code>bonus_multiplier / multiplier</code> aceptan <code>0</code> — el bono se acredita disponible (retirable) al instante, sin objetivo de apuestas.
1.5	2026-07-29	Orden de consumo del saldo (2.8): jugando se gasta real bloqueado → bono → disponible; <code>wagering.available</code> refleja ese orden. En modo "cada saldo con su propia plata", lo jugado con el disponible no progresa objetivos.
1.4	2026-07-29	Bono de monto fijo: el depósito acepta <code>bonus_amount</code> (excluyente con <code>bonus_percent</code>) y nuevo endpoint <code>POST /players/{username}/bonus</code> (bono suelto , sección 2.9) con idempotencia por <code>reference</code> . Error nuevo <code>bonus_out_of_range</code> (límites min/max del operador). Modo configurable "cada saldo con su propia plata" y regla de pisado documentados en 2.8.
1.3	2026-07-28	Rollover y Bonos (sección 2.8): el depósito acepta <code>multiplier</code> , <code>bonus_percent</code> y <code>bonus_multiplier</code> ; <code>GET /players/{username}</code> expone el desglose <code>wagering</code> (disponible/bloqueado/bono + progreso); el retiro respeta el disponible (<code>rollover_locked</code>). Errores nuevos <code>rollover_locked</code> y <code>feature_disabled</code> .
1.2	2026-07-13	<code>/session</code> : la contraseña pasa a ser opcional (login único solo con API key + username). Aclaración de que <code>amount / balance</code> van en pesos (no centavos), con decimales.
1.1	2026-07-11	Alta autoservicio por agente: la key se crea desde el panel del agente (Clientes de API); el creador es el padre de los jugadores y su saldo respalda las cargas.

Versión	Fecha	Cambios
1.0	2026-07-11	Primera versión: alta, validación, contraseña, saldo, depósito/retiro, login único.